



Work with strings

We have covered a lot of material so far. You learned about functions and conditional statements, and loops. Now, we'll learn more about different data types in Python, beginning with strings. Recall that a string is a sequence of characters and punctuation that contains textual information. This is an immutable data type, which means the values can never be altered or updated. Even though strings are immutable, we can still do a lot with them. For instance, we can concatenate them.

To concatenate, means to link or join together, so concatenating strings is making a single longer string from two or more shorter ones. To concatenate strings in Python, we simply use the addition operator. If we have two strings, "Hello" and "world," we can join them by adding them together. The result is a single string, but it's also a single word.

This is because blank spaces, or white spaces, as they're known in computer programming, count as their own characters. If you want a whitespace between your concatenated strings, one of the strings must contain a white space. Or, you must add a third string between them that contains just a whitespace.

The same rules apply when using variables that point to strings. If "Hello" is assigned to "greeting one," and "world" is assigned to "greeting two," we can concatenate the strings by adding the two variables. We can also multiple strings using the multiplication

operator. "Danger" times three, equals "Danger! Danger! Danger!" However, we cannot divide, or subtract strings, in trying to do so, will throw an error.

As you know, some characters are reserved for specific purposes when working with strings. For example, quotation marks are used to indicate the beginning and end of strings. But what if we want our string to contain quotation marks? There are two ways we can approach this. The first way takes advantage of the fact that strings can be written with either single quotes or double quotes. If you want to include double quotation marks in your string, use single quotation marks to begin and end your string, and vice-versa. The second way is to use the backslash, which functions as an escape character.

An escape character changes the typical behavior of the characters that follow it. In this case, the typical behavior of quotation marks is to begin or end the string, but if we proceed each with a backslash, they'll behave as regular punctuation marks in the string. The backslash character is useful as an escape character for other special characters in strings, too. For example, backslash-N is a special character combination that used to indicate a new line when printing a string. But if you want to include backslash-N as characters in your string when you print it, you must precede the combination with an initial backslash. Moving on, we can also iterate over strings with loops. In this example, we use a for loop to iterate over each letter of the word Python, and print the letter, plus the letters U-T.

These are just a few of the ways to work with strings. As a data professional, you'll often work with strings when analyzing data. Coming up, we'll cover more useful operations that we can perform over strings.

Concatenate

To link or join together

Escape character

A character that changes the typical behavior of the characters that follow it

Question

×

Fill in the blank: _____ strings makes a single longer string from two or more shorter ones.

- ☐ Branching
- ☒ Concatenating
- ☐ Converting
- ☐ Iterating

✓ Correct

Concatenating strings makes a single longer string from two or more shorter ones. To concatenate means to link or join together.

Continue

Skip